



String slicing

Recently you learned the basics of strings. In this video, you'll add to your knowledge by exploring a new way to work with strings: slicing. But before you learn about slicing, you'll need some background information about how Python works. As a reminder, an object is iterable if you can sequence through all of its values or items. Indexing is Python's way of letting us refer to individual items within an iterable by their relative position. Indexing is a very important part of Python 'cause it allows us to select, filter, edit and manipulate data, and it opens up many possibilities to the data professional. By the way, indexing isn't just for strings.

It also works on many other data types, like lists, tuples, and others, as long as they're iterable. You'll learn about these other data types soon. Python uses zero-based indexing. That means that the first element of a sequence is indexed at zero. With strings, indexing works by interpreting a string as a sequence of characters, where each character has a numbered slot. If you're reading from left to right, the first character is located at slot zero. The second character is located at slot one.

And the third at slot two, and so on. Indexing lets us slice strings to create smaller strings, or substrings. Here's an example that many data professionals have experience with: A column in a data set contains employee salary information. In the same field,

there will be both strings and integers: the currency symbol and the salary amount. In this case, Python would automatically interpret the mix of data types as a string data type. This is often a problem because we usually want values that represent money to behave like numbers, so we can perform mathematical operations on them. If they're strings, we can't do that.

So, to fix the problem, slicing helps us remove the non-numeric characters, like the dollar sign, from the string. In this case, we could drop the character at the zero index of each value in the salary column. This gives us salary information without the currency prefix. Let's explore some ways of working with indices. One useful tool is the index method. Index is a string method that outputs the index number of a character in a string. Remember, a method is a function that applies to a variable.

We can call it by following the variable with a dot. We use the index method to identify the location of a character or substring in a string. Here we have a variable called "pets," which has been assigned the string "cats and dogs." We use the index method by attaching it to the "pets" variable with a dot. In its parentheses, we enter the character we want the index of. Let's find S. When we run the cell, the computer returns the number three.

This means that index three of our string contains the letter S. What if there's more than one of the same character or substring? Here we know that there are two Ss in "cats and dogs," but only one index returns to us: three. That's because the index method just returns the first position that matches. And, if you search for a substring that is not there, say Z, you'll get a ValueError, because the substring is not found. Additionally, we

can also use an index number to find a specific character in that position. For example, we'll assign the string "Jolene" to a variable called "name."

By placing the index number in brackets after the variable, we can access the character at that position. "name" at index zero is J, And "name" at index five is E. What happens if we put six instead? We'd get an `IndexError`, indicating that the string index is out of range. You can access the last character of a string even when you don't know how long the string is by using negative indices. Let's consider an example. We don't know the length of this string, but it doesn't matter.

Since it isn't super efficient to count each character out, we can reference it by starting from the last position with a negative index. By using the index negative one, we get the last character, an exclamation point. And if we use the index negative two, we get the second to last character, A. Now that we've gone over the fundamentals of indexing, let's do some slicing. A string slice is a portion of a string. It's also known as a substring. String slices can contain more than one character.

Here's an example of how we slice a string of the word "orange." Let's start by putting some index numbers inside square brackets and separating the numbers by a colon. This defines the range of characters in the new slice. We'll go from index one up to index four. The closing index is not included in the range that's returned to us, so this would capture indices one, two and three. And we've extracted a slice that contains the characters that correspond to these indices, R-A-N. We can also use slice notation with just one of the two indices.

Omitting the first number in the range implies that the range begins at zero. So if the string is "pineapple," and we indicate our slice using "colon, four," we'll capture the first

four letters: "pine." Similarly, if we slice using "four, colon", we'll capture everything beginning with index four all the way to the end: "apple." Great! We have one more thing to learn. Sometimes data professionals want to check whether or not a substring is contained in a string. To check whether or not a substring is contained in a string, use the keyword `in`.

Let's find out if "banana" is in the string contained in our "fruit" variable. It's false. There is no banana in our pineapple, but is there "apple?" Let's check. Yes, "apple" is a substring of "pineapple." So the computer returns a value of `True`. Confirming whether a substring is contained in a string is a common practice in all kinds of data careers. I encourage you to take some time to go through the steps again on your own. The more you apply what you learn, the more comfortable you will become.

Indexing

A way to refer to the individual items within an iterable by their relative position

Indexing can be used on:

- strings
- lists
- tuples
- most other iterable data types

`index()`

A string method that outputs the index number of a character in a string

Question



Python uses one-based indexing.

- ☐ True
- ☒ False

✓ **Correct**

Python uses zero-based indexing. This means that the first element of a sequence is indexed as zero. With strings, indexing works by interpreting a string as a sequence of characters, where each character has a numbered slot.

Continue

Skip

String slice

A portion of a string, also known as a substring, that can contain more than one character

To check whether or not a substring is contained in a string, use the keyword `in`.